

Gradient-based hyperparameter optimization

MIPT

2025

Model selection: coherent inference

First level: select optimal parameters:

$$\mathbf{w} = \arg \max \frac{p(\mathcal{D}|\mathbf{w})p(\mathbf{w}|\mathbf{h})}{p(\mathcal{D}|\mathbf{h})},$$

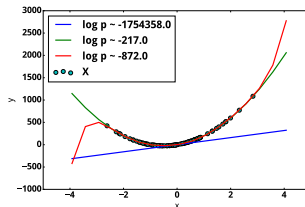
Second level: select optimal model (hyperparameters).

Evidence:

$$p(\mathcal{D}|\mathbf{h}) = \int_{\mathbf{w}} p(\mathcal{D}|\mathbf{w})p(\mathbf{w}|\mathbf{h})d\mathbf{w}.$$



Model selection scheme



Example: polynomials

Hyperparameters

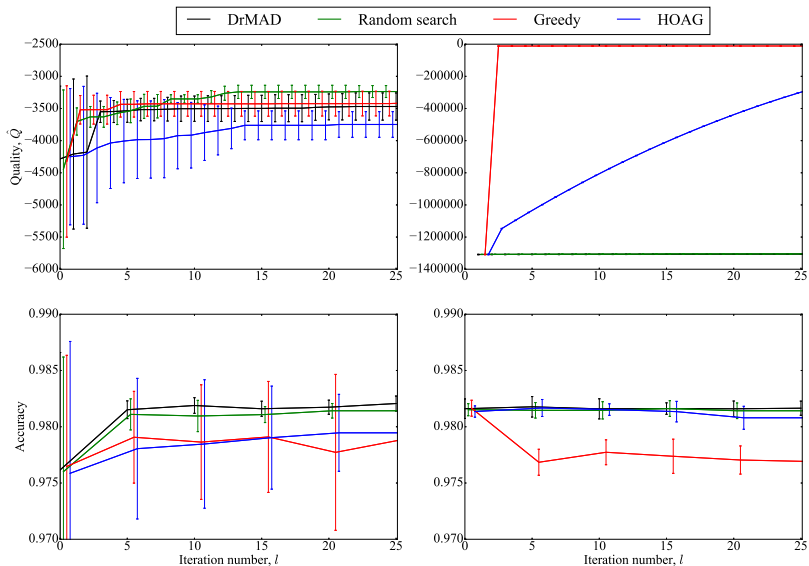
Definition

Prior for parameters $\mathbf{w}$ and structure Γ of the model $\mathbf{f}$ is a distribution $p(\mathbf{W}, \Gamma | \mathbf{h}) : \mathbb{W} \times \mathbb{\Gamma} \times \mathbb{H} \rightarrow \mathbb{R}^+$, where $\mathbb{W}$ is a parameter space, $\mathbb{\Gamma}$ is a structure space.

Definition

Hyperparameters $\mathbf{h} \in \mathbb{H}$ of the models are the parameters of $p(\mathbf{w}, \Gamma | \mathbf{h})$ (parameters of prior $\mathbf{f}$).

Experiment: MNIST



Experiment: MNIST

Adding Gaussian noise $\mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$:



Without noise



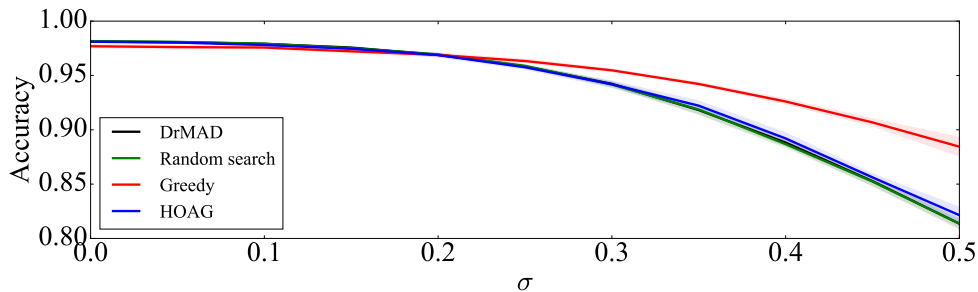
$\sigma = 0.1$



$\sigma = 0.25$

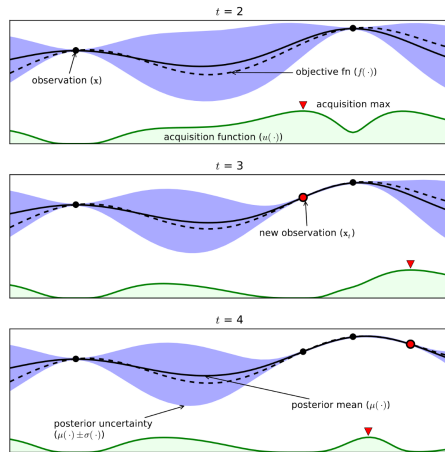


$\sigma = 0.5$



Non-gradient based methods: cons

- Need to run multiple times to get optimal hyperparameter values
- Computational complexity:
 - ▶ GP: $O(m^3)$, m is the number of observations.
 - ▶ TPE: $O(m \log m)$



Shahriari et. al, 2016. GP example.

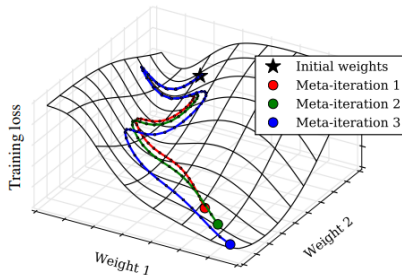
Gradient methods

Idea: Optimize hyperparameters using the full parameter optimization trajectory.

Pros:

- Hyperparameter optimization will consider the features of the parameter optimization.
- The complexity is linear on the number of hyperparameters.

Cons: the computational cost is very expensive. Also can be problems with numerical precision of reverse method.



Maclaurin et. al, 2015. Example.

Problem statement and notations

Given:

- $\mathbf{w} \in \mathbb{R}^N$: model parameters (or variational parameters θ);
- $\lambda \in \mathbb{R}^H$: hyperparameters;
- differentiable train and validation losses $L_{\text{train}}(\mathbf{w}, \lambda), L_{\text{val}}(\mathbf{w}, \lambda)$.

Hyperparameter optimization can be considered as a bi-level optimization problem:

$$\lambda^* = \arg \min_{\lambda \in \mathbb{R}^H} L_{\text{val}}(\mathbf{w}^*, \lambda),$$

$$\text{s. t. } \mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{R}^N} L_{\text{train}}(\mathbf{w}, \lambda).$$

To use gradient-based optimization we need to find $\nabla_{\lambda} L_{\text{val}}(\mathbf{w}^*(\lambda), \lambda)$.

Bi-level optimization: simple example

Classification problem with l_2 -regularization:

$$\lambda^* = \arg \min_{\lambda^* \in \mathbb{R}^H} \text{CrossEntropy}(\mathbf{f}(\mathbf{X}_{\text{val}}, \mathbf{w}), \mathbf{y}_{\text{val}})$$

$$\text{s. t. } \mathbf{w}^* = \arg \min_{\mathbf{w}^* \in \mathbb{R}^N} \text{CrossEntropy}(\mathbf{f}(\mathbf{X}_{\text{train}}, \mathbf{w}), \mathbf{y}_{\text{train}}) + \lambda \|\mathbf{w}\|^2.$$

Bi-level optimization: simple example

In general, λ can be presented in both problems:

$$\lambda^* = \arg \min_{\lambda^* \in \mathbb{R}^H} \text{CrossEntropy}(\mathbf{f}(\mathbf{X}_{\text{val}}, \mathbf{w}), \mathbf{y}_{\text{val}}) + \mathbf{G}(\lambda)$$

$$\text{s. t. } \mathbf{w}^* = \arg \min_{\mathbf{w}^* \in \mathbb{R}^N} \text{CrossEntropy}(\mathbf{f}(\mathbf{X}_{\text{train}}, \mathbf{w}), \mathbf{y}_{\text{train}}) + \lambda \|\mathbf{w}\|^2.$$

Problem statement: gradient-based optimization

Definition

An SGD operator T is an operator providing η steps of optimization:

$$\hat{\mathbf{w}} = T \circ T \circ \dots \circ T(\mathbf{w}_0, \mathbf{h}) = T^\eta(\mathbf{w}_0, \mathbf{h}), \quad (1)$$

where

$$T(\mathbf{w}, \mathbf{h}) = \mathbf{w} - \lambda \nabla L_{\text{train}}(\mathbf{w}, \mathbf{h})|_{\hat{\mathcal{D}}},$$

λ is a learning rate, $\mathbf{w}_0$ is an initial value of $\mathbf{w}$, $\hat{\mathcal{D}}$ is a random subset of $\mathcal{D}$.

Rewrite the optimization problem:

$$\mathbf{h}^* = \arg \max_{\mathbf{h} \in \mathbb{H}} L_{\text{val}}(T^\eta(\mathbf{w}_0, \mathbf{h})).$$

Gradient-Based Optimization of Hyperparameters, Bengio 2000

Simple case with quadratic train loss:

$$L_{\text{train}} = \mathbf{a}(\lambda) + \mathbf{b}(\lambda)^T \mathbf{w} + \frac{1}{2} \mathbf{w}^T \mathbf{H}(\lambda) \mathbf{w}.$$

Solution for the inner problem:

$$\nabla_{\mathbf{w}} L_{\text{train}} = \mathbf{b} + \mathbf{H}(\lambda) \mathbf{w} = 0, \quad \mathbf{w}^*(\lambda) = -\mathbf{H}^{-1}(\lambda) \mathbf{b}(\lambda).$$

Assuming L_{val} depends on λ only via $\mathbf{w}$:

$$\underbrace{\nabla_{\lambda} L_{\text{val}}(\mathbf{w}^*(\lambda), \lambda)}_{\text{gradient we need to compute}} = \underbrace{(\nabla_{\lambda} \mathbf{w}^*(\lambda))}_{\text{best-response Jacobian}}^T \nabla_{\mathbf{w}} L_{\text{val}}, \quad \nabla_{\lambda} w_i = - \sum_{j=1}^H \frac{\partial H_{ij}^{-1}}{\partial \lambda} b_j - \sum_{j=1}^H H_{ij}^{-1} \frac{\partial b_j}{\partial \lambda}.$$

The major problem is computing first term $\frac{\partial H_{i,j}^{-1}}{\partial \lambda}$.

Gradient-Based Optimization of Hyperparameters, Bengio 2000

$$\underbrace{\nabla_{\lambda} L_{\text{val}}(\mathbf{w}^*(\lambda), \lambda)}_{\text{gradient we need to compute}} = \underbrace{(\nabla_{\lambda} \mathbf{w}^*(\lambda))^\top}_{\text{best-response Jacobian}} \nabla_{\mathbf{w}} L_{\text{val}}, \quad \nabla_{\lambda} w_i = - \sum_{j=1}^H \frac{\partial H_{ij}^{-1}}{\partial \lambda} b_j - \sum_{j=1}^H H_{ij}^{-1} \frac{\partial b_j}{\partial \lambda}.$$

2 ways to solve the problem:

- Naive approach using gradient over inverse Hessian matrix: $O(N^5)$.
- Solve the original equation $\nabla_{\mathbf{w}} L_{\text{train}} = \mathbf{b} + \mathbf{H}(\lambda)\mathbf{w}$ and backpropagate over the solution: $O(N^3)$.

General case, Bengio 2000

In general case we don't have an analytical solution for the inner problem and need to optimize it using numerical methods.

Implicit function theorem: the function $\mathbf{w}(\lambda)$ exists and differentiable near optimum $\mathbf{w}^*(\lambda)$:

$$\mathbf{F}(\mathbf{w}^*(\lambda), \lambda) = \nabla_{\mathbf{w}} L_{\text{train}}|_{\lambda, \mathbf{w}^*(\lambda)} = 0.$$

Differentiate $\mathbf{F}$ by λ at $\lambda, \mathbf{w}^*(\lambda)$:

$$\frac{\partial \mathbf{F}}{\partial \mathbf{w}} \frac{\partial \mathbf{w}}{\partial \lambda} + \frac{\partial \mathbf{F}}{\partial \lambda} = 0 \rightarrow$$

$$(\nabla_{\lambda} \mathbf{w}(\lambda))^T \nabla_{\mathbf{w}}^2 L_{\text{train}} + \nabla_{\mathbf{w}, \lambda} L_{\text{train}} = 0.$$

We get a general closed form expression for the gradient:

$$\begin{aligned} \nabla_{\lambda} L_{\text{val}}(\mathbf{w}^*(\lambda), \lambda) &= \nabla_{\lambda} L_{\text{val}} + (\nabla_{\lambda} \mathbf{w}(\lambda))^T \nabla_{\mathbf{w}} L_{\text{val}} \\ &= \underbrace{\nabla_{\lambda} L_{\text{val}}}_{\text{Direct gradient}} - \left(\nabla_{\mathbf{w}, \lambda}^2 L_{\text{train}} \right)^T \underbrace{\left(\nabla_{\mathbf{w}}^2 L_{\text{train}} \right)^{-1}}_{\text{Inverse Hessian}} \nabla_{\mathbf{w}} L_{\text{val}}. \end{aligned}$$

Issues

$$\nabla_{\lambda} L_{\text{val}}(\mathbf{w}^*(\lambda), \lambda) = \underbrace{\nabla_{\lambda} L_{\text{val}}}_{\text{Direct gradient}} - (\nabla_{\mathbf{w}, \lambda}^2 L_{\text{train}})^{\top} \underbrace{(\nabla_{\mathbf{w}}^2 L_{\text{train}})^{-1}}_{\text{Inverse Hessian}} \nabla_{\mathbf{w}} L_{\text{val}}.$$

- $O(N^3)$ is better than $O(N^5)$, but still impractical for large-scale problems.
- We optimize hyperparameters λ for the converged values of $\mathbf{w}^*$.
Optimization of some hyperparameters therefore is impossible:
 - ▶ parameter initial value;
 - ▶ learning rate.

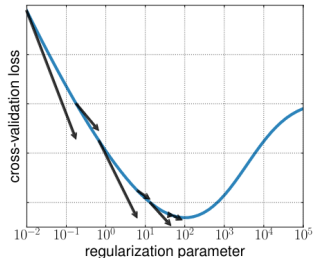
Hyperparameter optimization with approximate gradient, Pedregosa, 2016

Main idea: instead of computing $(\nabla_{\mathbf{w}}^2 L_{\text{train}})^{-1} \nabla_{\mathbf{w}} L_{\text{val}}$ we find a vector $\mathbf{v}$ that $\nabla_{\mathbf{w}}^2 L_{\text{train}} \mathbf{v} \approx \nabla_{\mathbf{w}} L_{\text{val}}$.

In a loop:

- Solve the inner optimization problem up to tolerance ε : $|\mathbf{w}(\lambda) - \mathbf{w}^*| < \varepsilon$.
- Solve the linear system $\nabla_{\mathbf{w}}^2 L_{\text{train}} \mathbf{v} = \nabla_{\mathbf{w}} L_{\text{val}}$ w.r.t. $\mathbf{v}$ up to tolerance ε .
- Compute approximate gradient for λ as

$$\nabla_{\lambda} L_{\text{val}}(\mathbf{w}(\lambda), \lambda) \approx \nabla_{\lambda} L_{\text{val}} - (\nabla_{\mathbf{w}, \lambda}^2 L_{\text{train}})^T \mathbf{v}$$



HOAG, method analysis

Regularity conditions

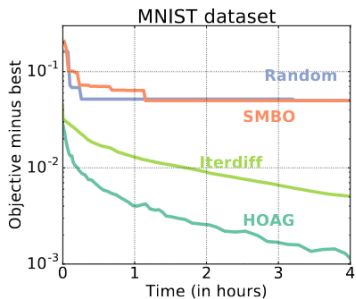
- First derivative of L_{train} and second derivatives of L_{val} are Lipschitz smooth;
- Hessian $\nabla_{\mathbf{w}}^2 L_{\text{train}}$ is invertible;
- Hyperparameter domain is convex.

HOAG properties

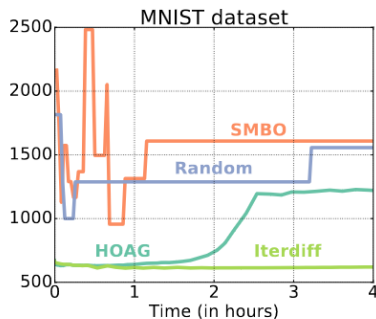
- For large number of iterations k the error in the gradient is bounded by a constant factor of ε_k : $\|\nabla_{\lambda} L_{\text{val}}(\mathbf{w}(\lambda), \lambda) - \widetilde{\nabla}_{\lambda} L_{\text{val}}(\mathbf{w}(\lambda), \lambda)\| = O(\varepsilon_k)$;
- If $\sum_{k=0}^{\infty} \varepsilon_k < \infty$, then $\lim_{k \rightarrow \infty} \|\widetilde{\nabla}_{\lambda} L_{\text{val}}(\mathbf{w}(\lambda), \lambda)\| = 0$ when performing gradient descent.

HOAG, baseline comparison

Model: logistic regression trained on MNIST (7840 hyperparameters).



Validation dataset



Test dataset

We can see:

- The only methods worked well are gradient-based methods (HOAG and iterdiff);
- There is an overfitting on validation part of the dataset.

Approximation using Neumann series, Lorraine et al., 2020 ¹

Recap: Neumann series

Given an operator T , a Neumann series for it: $\sum_{i=0}^{\infty} T^i$.

If T is bounded, linear and its Neumann series converges, then

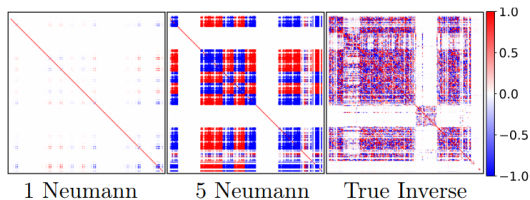
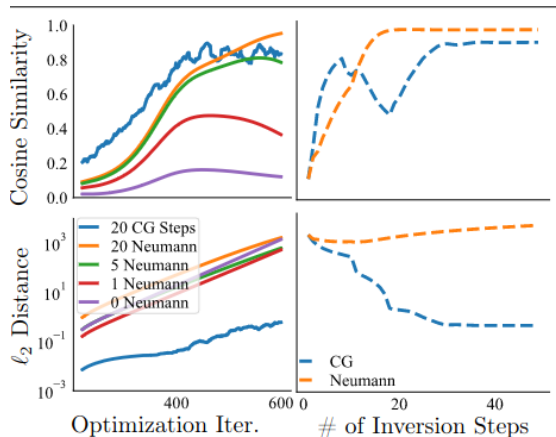
$$(I - T)^{-1} = \sum_{i=0}^{\infty} T^i.$$

We approximate inverse Hessian $(\nabla_{\mathbf{w}}^2 L_{\text{train}})^{-1}$ using K terms of the series:

$$(\nabla_{\mathbf{w}}^2 L_{\text{train}})^{-1} \approx \sum_{i=0}^K I - (\nabla_{\mathbf{w}}^2 L_{\text{train}})^i,$$

¹See Victor Pankratov's talk, 2021

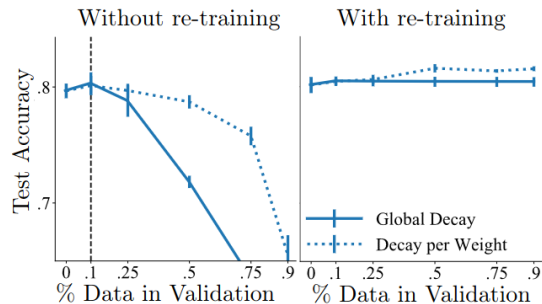
Inverse Hessian approximation quality



Inverse Hessian approximation for 1-layer NN,
Boston dataset

Inverse Hessian approximation using HOAG-like
method and Neumann series

Large-scale experiments



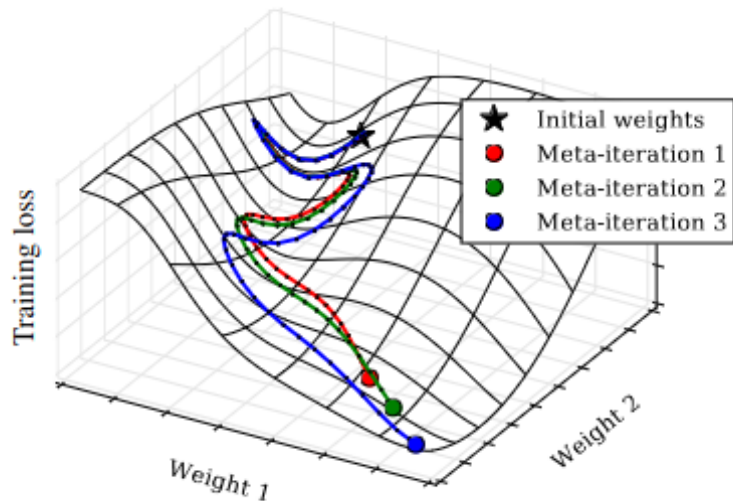
Logistic regression, MNIST

Method	Validation	Test	Time(s)
Grid Search	97.32	94.58	100k
Random Search	84.81	81.46	100k
Bayesian Opt.	72.13	69.29	100k
STN	70.30	67.68	25k
No HO	75.72	71.91	18.5k
Ours	69.22	66.40	18.5k
Ours, Many	68.18	66.14	18.5k

RNN with DropConnect coefficient optimization: up to 1.7M hyperparameters

RMAD, Maclaurin et al., 2015

Why not just propagate over the full optimization trajectory?



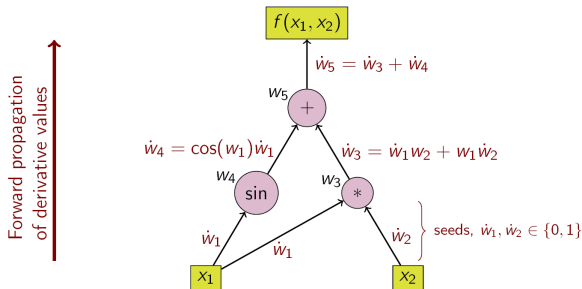
Forward-mode differentiation

Main idea:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial w_{n-1}} \frac{\partial w_{n-1}}{\partial x} = \frac{\partial y}{\partial w_{n-1}} \left(\frac{\partial w_{n-1}}{\partial w_{n-2}} \frac{\partial w_{n-2}}{\partial x} \right) = \dots$$

Example (wiki):

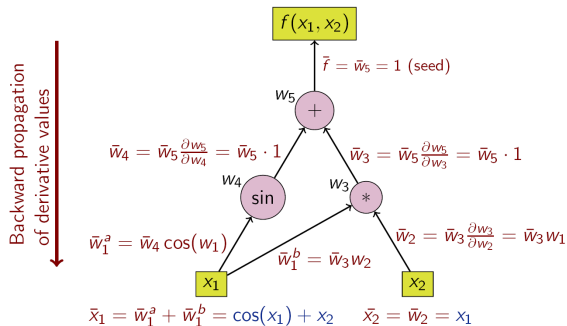
$$x_1 x_2 + \sin(x_1)$$



Reverse-mode differentiation

Idea:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial w_1} \frac{\partial w_1}{\partial x} = \left(\frac{\partial y}{\partial w_2} \frac{\partial w_2}{\partial w_1} \right) \frac{\partial w_1}{\partial x} = \dots$$



SGD with momentum and reverse-mode gradient of SGD

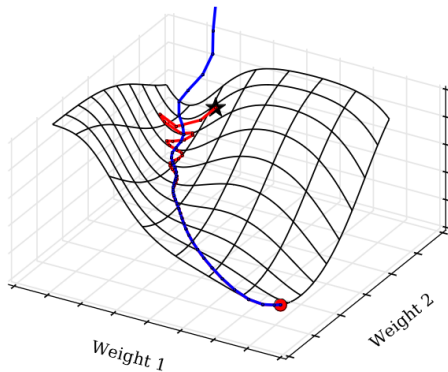
Stochastic Gradient Descent

```
1: input: initial  $\mathbf{x}_1$ , decays  $\beta$ , learning rates  $\alpha$ , loss  
   function  $L(\mathbf{x}, \theta, t)$   
2: initialize  $\mathbf{v}_1 = \mathbf{0}$   
3: for  $t = 1$  to  $T$  do  
4:    $\mathbf{g}_t = \nabla_{\mathbf{x}} L(\mathbf{x}_t, \theta, t)$            ▷ evaluate gradient  
5:    $\mathbf{v}_{t+1} = \beta_t \mathbf{v}_t - (1 - \beta_t) \mathbf{g}_t$    ▷ update velocity  
6:    $\mathbf{x}_{t+1} = \mathbf{x}_t + \alpha_t \mathbf{v}_t$            ▷ update position  
7: output trained parameters  $\mathbf{x}_T$ 
```

Reverse-Mode Gradient of SGD

```
1: input:  $\mathbf{x}_T, \mathbf{v}_T, \beta, \alpha$ , train loss  $L(\mathbf{x}, \theta, t)$ , loss  $f(\mathbf{x})$   
2: initialize  $d\mathbf{v} = \mathbf{0}, d\theta = \mathbf{0}, d\alpha_t = \mathbf{0}, d\beta = \mathbf{0}$   
3: initialize  $d\mathbf{x} = \nabla_{\mathbf{x}} f(\mathbf{x}_T)$   
4: for  $t = T$  counting down to 1 do  
5:    $d\alpha_t = d\mathbf{x}^T \mathbf{v}_t$   
6:    $\mathbf{x}_{t-1} = \mathbf{x}_t - \alpha_t \mathbf{v}_t$            ▷ downdate position  
7:    $\mathbf{g}_t = \nabla_{\mathbf{x}} L(\mathbf{x}_t, \theta, t)$            ▷ evaluate gradient  
8:    $\mathbf{v}_{t-1} = [\mathbf{v}_t + (1 - \beta_t) \mathbf{g}_t] / \beta_t$  ▷ downdate velocity  
9:    $d\mathbf{v} = d\mathbf{v} + \alpha_t d\mathbf{x}$   
10:   $d\beta_t = d\mathbf{v}^T (\mathbf{v}_t + \mathbf{g}_t)$   
11:   $d\mathbf{x} = d\mathbf{x} - (1 - \beta_t) d\mathbf{v} \nabla_{\mathbf{x}} \nabla_{\mathbf{x}} L(\mathbf{x}_t, \theta, t)$   
12:   $d\theta = d\theta - (1 - \beta_t) d\mathbf{v} \nabla_{\theta} \nabla_{\mathbf{x}} L(\mathbf{x}_t, \theta, t)$   
13:   $d\mathbf{v} = \beta_t d\mathbf{v}$   
14: output gradient of  $f(\mathbf{x}_T)$  w.r.t  $\mathbf{x}_1, \mathbf{v}_1, \beta, \alpha$  and  $\theta$ 
```

Naive approach: fails



Credits: https://www.robots.ox.ac.uk/seminars/Extra/2015_07_23_David_Duvenaud.pdf

Why reversible dynamics fails?

Update velocity step:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t - (1 - \beta) \mathbf{g}_t,$$

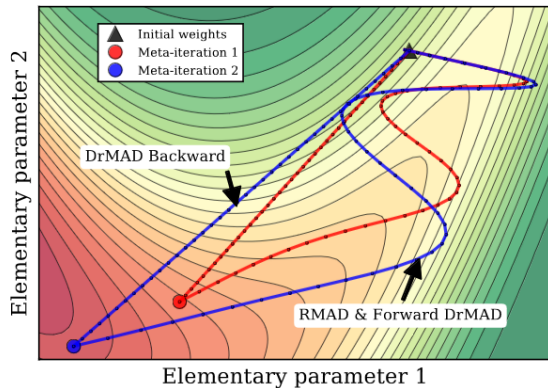
for the finite arithmetic this expression will destroy $\log_2 \beta$ bits of information.

Solution: make a scheme for storing destroyed bits of information to restore dynamics properly.

Conclusion: as a proof of concept, we really can restore the full trajectory, but this method requires momentum or similar techniques in optimization.

DRMAD, Fu et al., 2016

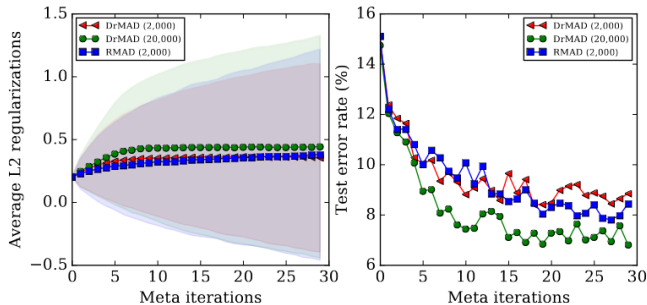
Instead of reversing dynamics replace it with linear trajectory of optimization.



DrMAD vs RMAD

Algorithm 2 Distilling reverse-mode automatic differentiation (DrMAD) of SGD.

- 1: **inputs:** initial parameter w_0 , learned parameter w_T , training loss l_{train} , validation loss l_{valid} , weight decay γ , learning rate α
 - 2: initialize $d\lambda \leftarrow 0$, $d\mathbf{w} \leftarrow \nabla_{\mathbf{w}} l_{valid}$, $\beta_{T-1} \leftarrow 1 - \frac{1}{T}$
 - 3: **for** $t = T - 1$ to 1 **do**
 - 4: $\mathbf{w}_{t-1} \leftarrow (1 - \beta_t)\mathbf{w}_0 + \beta_t\mathbf{w}_T$
 - 5: $\beta_{t-1} \leftarrow \beta_t - \frac{1}{T}$
 - 6: $d\mathbf{v} \leftarrow d\mathbf{v} + \alpha d\mathbf{w}$
 - 7: $d\lambda \leftarrow d\lambda - (1 - \gamma)d\mathbf{v}\nabla_{\lambda}\nabla_{\mathbf{w}} l_{train}$
 - 8: **end for**
 - 9: **output:** gradient of l_{valid} w.r.t. λ
-



T1-T2, Luketina et al., 2016

Main idea: instead of using full optimization trajectory, use just a one step.

The optimization can be considered as the following:

- Perform an optimization step for parameters $\mathbf{w}_{t+1}$
- Perform virtual next step optimization for parameters $\mathbf{w}_{t+1}(\mathbf{w}_t, \lambda)$
- Calculate hyperparameter gradients from virtually computed $\mathbf{w}_{t+1}$ as a function from $\mathbf{w}_t$:

$$\nabla_{\lambda} L_{\text{val}}(\mathbf{w}^*(\lambda), \lambda) \approx \nabla_{\lambda} L_{\text{val}}(\mathbf{w}_{t+1}(\mathbf{w}_t, \lambda), \lambda).$$

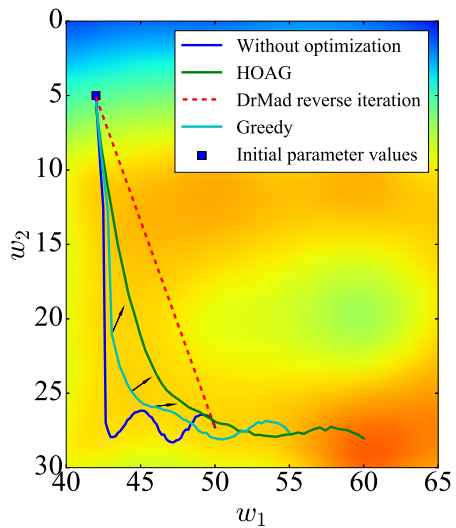
T1-T2, method properties

- The method works in a greedy way, there is no strict guarantees for convergence or optimality.
- The closed form of the hyperparameter gradient:

$$\nabla_{\mathbf{w}} L_{\text{val}} \nabla_{\lambda, \mathbf{w}} L_{\text{train}}.$$

This expression coincides with general closed form when $(\nabla_{\mathbf{w}}^2 L_{\text{train}})^{-1} = \mathbf{I}$.

- The method complexity is $O(|\mathbf{w}| \cdot |\lambda|)$, but can be improved by using finite difference approximation $O(|\mathbf{w}| + |\lambda|)$, (Liu 2019 et al).
- We can go further and optimize hyperparameter even without making a virtual step, (Liu 2019 et al).



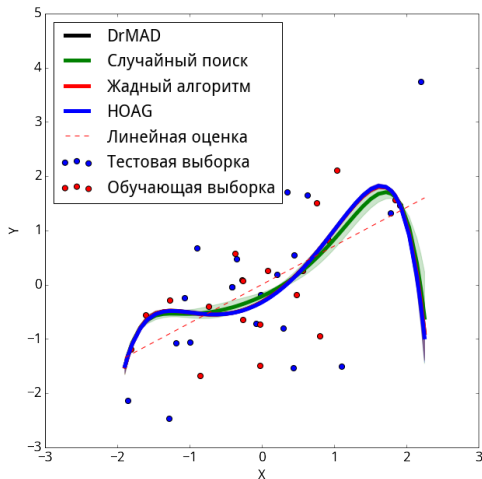
Algorithm comparison

Algorithm	+	-
Random search	Easy to implement	Curse of dimensionality
Greedy optimization	Easy to implement, can be run inside common gradient optimization loop	Non-optimality.
HOAG	Fast convergence.	Results quality depends on the the linear system solution precision $\mathbf{H}(\theta)\lambda = \nabla_{\theta}Q(\theta, \mathbf{h})$.
DrMAD	Considers optimization features. Can be used for metaparameter optimization.	Not very robust. Strong assumptions about linearity of the optimization trajectory.

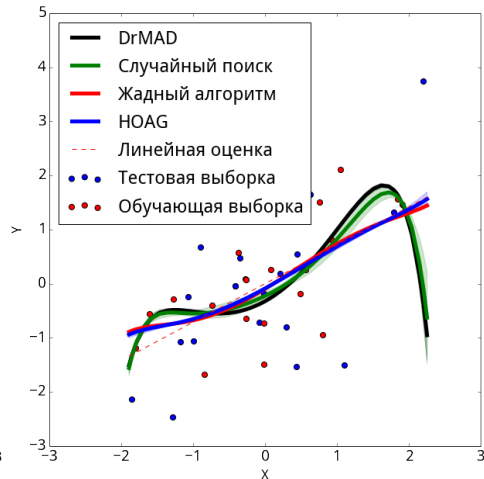
Overall comparison

Method	Steps	Eval.	Hypergradient Approximation
Exact IFT	∞	$\mathbf{w}^*(\lambda)$	$\frac{\partial \mathcal{L}_Y}{\partial \lambda} - \frac{\partial \mathcal{L}_Y}{\partial \mathbf{w}} \times \left[\frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \mathbf{w}} \right]^{-1} \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \lambda} \Big _{\mathbf{w}^*(\lambda)}$
Unrolled Diff. [Maclaurin et al., 2015]	i	$\mathbf{w}_0$	$\frac{\partial \mathcal{L}_Y}{\partial \lambda} - \frac{\partial \mathcal{L}_Y}{\partial \mathbf{w}} \times \sum_{j \leq i} \left[\prod_{k < j} I - \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \mathbf{w}} \Big _{\mathbf{w}_{i-k}} \right] \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \lambda} \Big _{\mathbf{w}_{i-j}}$
L -Step Truncated Unrolled Diff.	i	$\mathbf{w}_L$	$\frac{\partial \mathcal{L}_Y}{\partial \lambda} - \frac{\partial \mathcal{L}_Y}{\partial \mathbf{w}} \times \sum_{L \leq j \leq i} \left[\prod_{k < j} I - \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \mathbf{w}} \Big _{\mathbf{w}_{i-k}} \right] \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \lambda} \Big _{\mathbf{w}_{i-j}}$
Larsen et al. [1996]	∞	$\widehat{\mathbf{w}}^*(\lambda)$	$\frac{\partial \mathcal{L}_Y}{\partial \lambda} - \frac{\partial \mathcal{L}_Y}{\partial \mathbf{w}} \times \left[\frac{\partial \mathcal{L}_T}{\partial \mathbf{w}} \frac{\partial \mathcal{L}_T^T}{\partial \mathbf{w}} \right]^{-1} \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \lambda} \Big _{\widehat{\mathbf{w}}^*(\lambda)}$
Bengio [2000]	∞	$\widehat{\mathbf{w}}^*(\lambda)$	$\frac{\partial \mathcal{L}_Y}{\partial \lambda} - \frac{\partial \mathcal{L}_Y}{\partial \mathbf{w}} \times \left[\frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \mathbf{w}} \right]^{-1} \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \lambda} \Big _{\widehat{\mathbf{w}}^*(\lambda)}$
$T1 - T2$ [27]	1	$\widehat{\mathbf{w}}^*(\lambda)$	$\frac{\partial \mathcal{L}_Y}{\partial \lambda} - \frac{\partial \mathcal{L}_Y}{\partial \mathbf{w}} \times [I]^{-1} \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \lambda} \Big _{\widehat{\mathbf{w}}^*(\lambda)}$
Ours	i	$\widehat{\mathbf{w}}^*(\lambda)$	$\frac{\partial \mathcal{L}_Y}{\partial \lambda} - \frac{\partial \mathcal{L}_Y}{\partial \mathbf{w}} \times \left(\sum_{j \leq i} \left[I - \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \mathbf{w}} \right]^j \right) \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \lambda} \Big _{\widehat{\mathbf{w}}^*(\lambda)}$
Conjugate Gradient (CG) $\approx$	-	$\widehat{\mathbf{w}}^*(\lambda)$	$\frac{\partial \mathcal{L}_Y}{\partial \lambda} - \left(\arg \min_{\mathbf{x}} \left\ \mathbf{x} \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \mathbf{w}} - \frac{\partial \mathcal{L}_Y}{\partial \mathbf{w}} \right\ \right) \frac{\partial^2 \mathcal{L}_T}{\partial \mathbf{w} \partial \lambda} \Big _{\widehat{\mathbf{w}}^*(\lambda)}$
Hypernetwork	-	-	$\frac{\partial \mathcal{L}_Y}{\partial \lambda} + \frac{\partial \mathcal{L}_Y}{\partial \mathbf{w}} \times \frac{\partial \mathbf{w}_\phi^*}{\partial \lambda}$ where $\mathbf{w}_\phi^*(\lambda) = \arg \min_{\phi} \mathcal{L}_T(\lambda, \mathbf{w}_\phi(\lambda))$
Bayesian Optimization	-	-	$\frac{\partial \mathbb{E}[\mathcal{L}_Y^*]}{\partial \lambda}$ where $\mathcal{L}_Y^* \sim \text{Gaussian-Process}(\{\lambda_i, \mathcal{L}_Y(\lambda_i, \mathbf{w}^*(\lambda_i))\})$

Experiment: polynoms



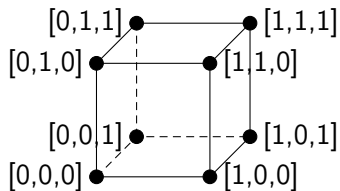
CV



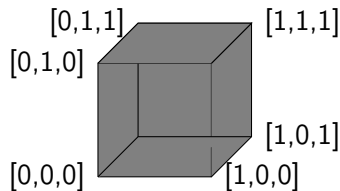
Evidence

Discrete parameter relaxation

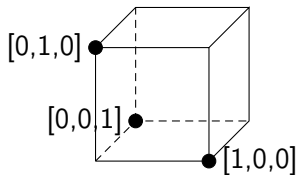
An example of restrictions for structure parameter γ , $|\gamma| = 3$.



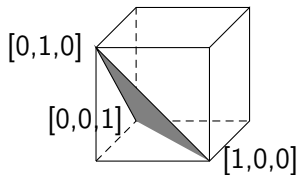
Cube vertices



Cube interior

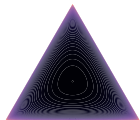


Simplex vertices



Simplex interior

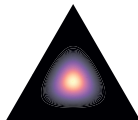
Discrete distribution: relaxation



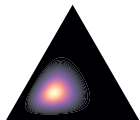
$$\bar{\alpha} = [1, 1, 1], t = 0.9$$



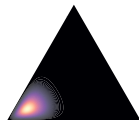
$$t = 1.0$$



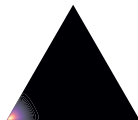
$$t = 10.0$$



$$\bar{\alpha} = [0.5, 0.25, 0.25], t = 30.0$$

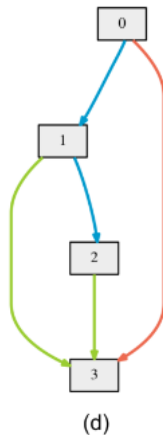
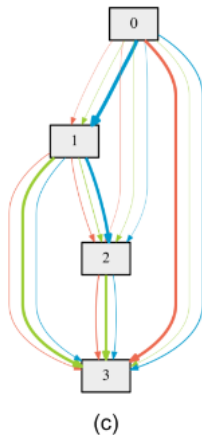
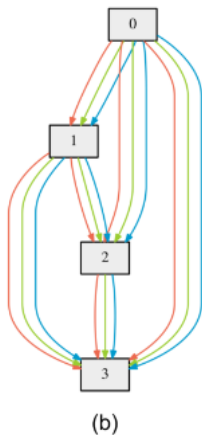
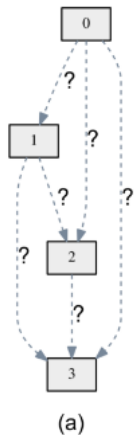


$$[0.75, 0.125, 0.125]$$



$$[0.9, 0.05, 0.05]$$

Application example: DARTS



Application example: DARTS

Applying chain rule to the approximate architecture gradient (equation 6) yields

$$\nabla_{\alpha} \mathcal{L}_{val}(w', \alpha) - \xi \nabla_{\alpha, w}^2 \mathcal{L}_{train}(w, \alpha) \nabla_{w'} \mathcal{L}_{val}(w', \alpha) \quad (7)$$

where $w' = w - \xi \nabla_w \mathcal{L}_{train}(w, \alpha)$ denotes the weights for a one-step forward model. The expression above contains an expensive matrix-vector product in its second term. Fortunately, the complexity can be substantially reduced using the finite difference approximation. Let ϵ be a small scalar² and $w^{\pm} = w \pm \epsilon \nabla_{w'} \mathcal{L}_{val}(w', \alpha)$. Then:

$$\nabla_{\alpha, w}^2 \mathcal{L}_{train}(w, \alpha) \nabla_{w'} \mathcal{L}_{val}(w', \alpha) \approx \frac{\nabla_{\alpha} \mathcal{L}_{train}(w^+, \alpha) - \nabla_{\alpha} \mathcal{L}_{train}(w^-, \alpha)}{2\epsilon} \quad (8)$$

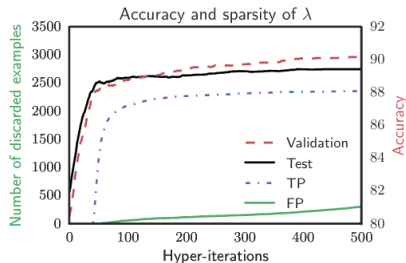
Evaluating the finite difference requires only two forward passes for the weights and two backward passes for α , and the complexity is reduced from $O(|\alpha||w|)$ to $O(|\alpha| + |w|)$.

First-order Approximation When $\xi = 0$, the second-order derivative in equation 7 will disappear. In this case, the architecture gradient is given by $\nabla_{\alpha} \mathcal{L}_{val}(w, \alpha)$, corresponding to the simple heuristic of optimizing the validation loss by assuming the current w is the same as $w^*(\alpha)$. This leads to some speed-up but empirically worse performance, according to our experimental results in Table 1 and Table 2. In the following, we refer to the case of $\xi = 0$ as the first-order approximation, and refer to the gradient formulation with $\xi > 0$ as the second-order approximation.

Data Hyper-cleaning

- Toy but illustrative example
- Given a very noisy training dataset with cleaning validation + test part, the task is to discard noisy examples from training part using sparsity constraints.
- Original experiment: MNIST, 5000 objects per train/validation/test, 2500 objects are corrupted.

	Accuracy %	F_1
Oracle	90.46	1.0000
Baseline	87.74	-
DH-1000	90.07	0.9137
DH-1500	90.06	0.9244
DH-2000	90.00	0.9211
DH-2500	90.09	0.9217



Other papers

- **Forward and Reverse Gradient-Based Hyperparameter Optimization, Franceschi et al., 2017**
 - ▶ The variation on the RMAD idea, could work with vanilla SGD
 - ▶ See Maksim Tyurikov's talk, 2023
- **Generalized Greedy Gradient-Based Hyperparameter Optimization, Yakovlev 2024**
 - ▶ See the pre-print at openreview

References

- Bengio, Yoshua. "Gradient-based optimization of hyperparameters." *Neural computation* 12.8 (2000): 1889-1900.
- Bishop C. M. *Pattern recognition // Machine learning*. – 2006. – T. 128. – №. 9.
- Bakhteev O. Y., Strijov V. V. Comprehensive analysis of gradient-based hyperparameter optimization algorithms // *Annals of Operations Research*. – 2020. – T. 289. – №. 1. – C. 51-65.
- Graves A. Practical variational inference for neural networks // *Advances in neural information processing systems*. – 2011. – T. 24.
- Bergstra et al., *Random Search for Hyper-Parameter Optimization*, 2012
- Dougal Maclaurin et. al, *Gradient-based Hyperparameter Optimization through Reversible Learning*, 2015
- Jelena Luketina et. al, *Scalable Gradient-Based Tuning of Continuous Regularization Hyperparameters*, 2016
- Jie Fu et. al, *DrMAD: Distilling Reverse-Mode Automatic Differentiation for Optimizing Hyperparameters of Deep Neural Networks*, 2016
- Fabian Pedregosa, *Hyperparameter optimization with approximate gradient*, 2016
- Bobak Shahriari et. al, *Taking the Human Out of the Loop: A Review of Bayesian Optimization*, 2016
- Liu H., Simonyan K., Yang Y. Darts: Differentiable architecture search // *arXiv preprint arXiv:1806.09055*. – 2018.
- Lorraine, J., Vicol, P., & Duvenaud, D. (2020, June). Optimizing millions of hyperparameters by implicit differentiation. In *International Conference on Artificial Intelligence and Statistics* (pp. 1540-1552). PMLR. (see also Victor Pankratov's slides, BMM-2021)
- Franceschi, Luca, et al. "Forward and reverse gradient-based hyperparameter optimization." *International Conference on Machine Learning*. PMLR, 2017.